

ROS roslaunch

Prof.: Dr. Oybek Eraliev

TA: Jasurbek Mamurov



Content:

- Roslaunch
- Rosaunch tags
- Hands-on exercises



Roslaunch

roslaunch = runs a single node (one at a time, one terminal per node)

roslaunch = runs one or more nodes defined in a ***.launch** file

Roslaunch also lets you:

- Change package parameters or node names
- Configure node namespaces to isolate communication groups
- Set environment variables (ROS_ROOT, ROS_PACKAGE_PATH)
- Load parameters from YAML files automatically
- Start roscore automatically if it is not already running
- Remap topic/service names between nodes

Basic usage:

```
$ roslaunch [package_name] [launch_file.launch]
```

```
$ roslaunch ros_tutorials_topic union.launch --screen
```

Roslaunch: Writing a .launch File

Create a launch folder inside your package, then write union.launch:

```
$ roscd ros_tutorials_topic
$ mkdir launch && cd launch
$ gedit union.launch
```

```
<!-- union.launch -->
<launch>
  <node pkg="ros_tutorials_topic" type="topic_publisher" name="topic_publisher1"/>
  <node pkg="ros_tutorials_topic" type="topic_subscriber" name="topic_subscriber1"/>
  <node pkg="ros_tutorials_topic" type="topic_publisher" name="topic_publisher2"/>
  <node pkg="ros_tutorials_topic" type="topic_subscriber" name="topic_subscriber2"/>
</launch>
```

Key **<node>** attributes:

pkg — Package name **type** — Executable filename **name** — Running node name (can differ from type)

Roslaunch

- Once you have created the roslaunch file, run union.launch as follows
`$ roslaunch your_pkg_name union.launch --screen`
- [Reference] How to output status when using roslaunch
 - When the roslaunch command runs several nodes, the output (info, error, etc.) of the running nodes is not displayed on the terminal screen and it makes debugging difficult. If you add the `--screen` option, the output of all nodes running on that terminal will be displayed on the terminal screen.

Roslaunch

- Execution result?

```
$ rosnode list
```

```
/topic_publisher1
```

```
/topic_publisher2
```

```
/topic_subscriber1
```

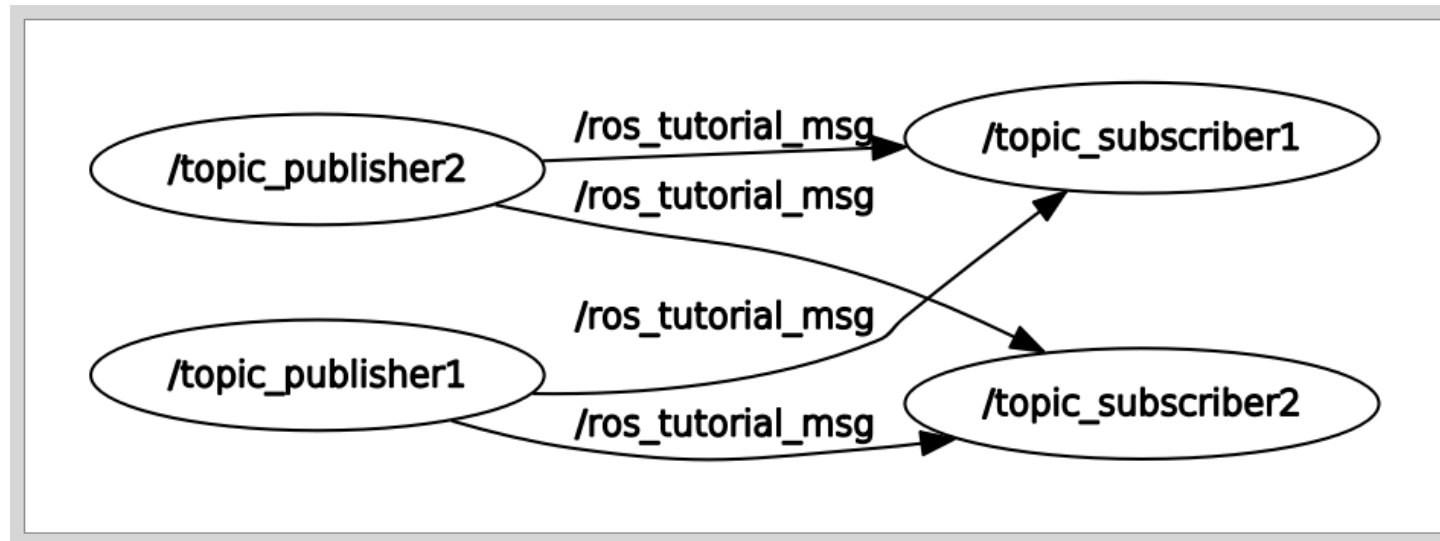
```
/topic_subscriber2
```

```
/rosout
```

- As a result, the topic_publisher node was renamed topic_publisher1 and topic_publisher2, and two nodes were running.
- The topic_subscriber node has also been renamed to topic_subscriber1 and topic_subscriber2.

Roslaunch

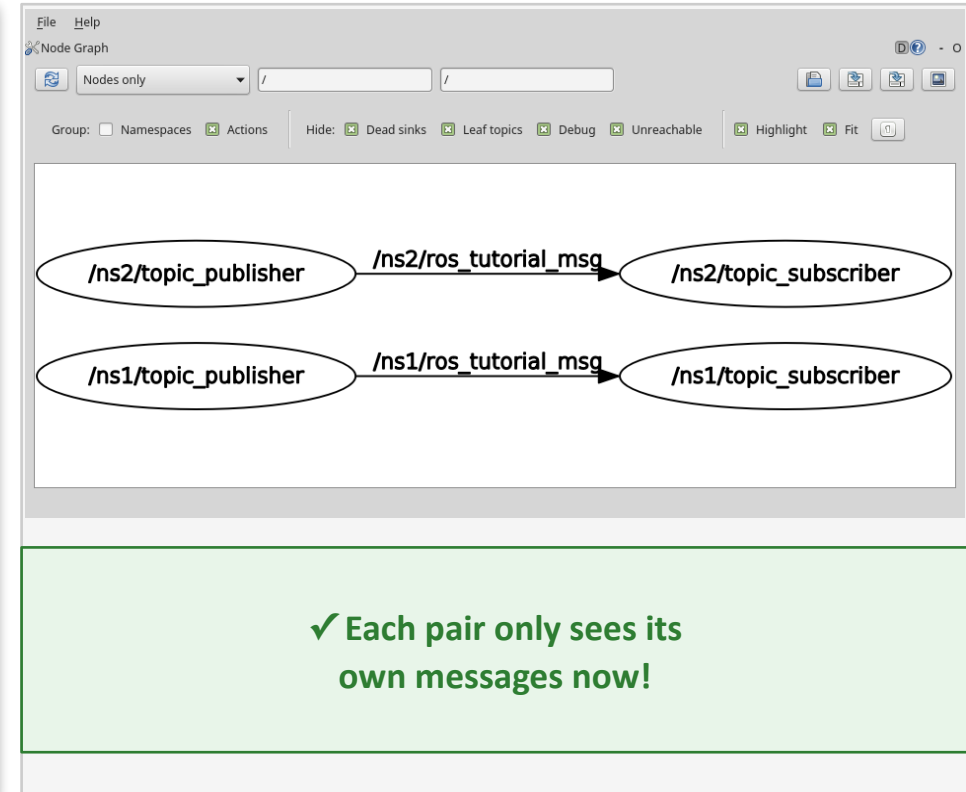
- The problem is that, unlike the initial intention that "two separate messages are communicated by driving two publisher nodes and subscriber nodes," rqt_graph shows that they are subscribing to each other's messages.
- This is because we simply changed the name of the node to be executed, but did not change the name of the message to be used.
- Let's fix this problem with another roslaunch namespace tag.



Roslaunch: Fixing with Namespaces

Use `<group ns="...">` to isolate each publisher-subscriber pair into its own namespace:

```
<launch>
  <group ns="ns1">
    <node pkg="ros_tutorials_topic"
type="topic_publisher" name="topic_publisher"/>
    <node pkg="ros_tutorials_topic"
type="topic_subscriber" name="topic_subscriber"/>
  </group>
  <group ns="ns2">
    <node pkg="ros_tutorials_topic"
type="topic_publisher" name="topic_publisher"/>
    <node pkg="ros_tutorials_topic"
type="topic_subscriber" name="topic_subscriber"/>
  </group>
</launch>
```



Roslaunch: <remap> Tag

<remap> lets you rename a topic, service, or action that a node uses — without modifying the node's source code. Perfect for connecting nodes that use different names.

```
<!-- Scenario: camera node publishes /image_raw
      but our processor expects /camera/image -->
<launch>
  <node pkg="usb_cam" type="usb_cam_node" name="camera">
    <remap from="/image_raw" to="/camera/image"/>
  </node>

  <!-- Now image_processor will receive /camera/image -->
  <node pkg="image_proc" type="processor" name="image_processor"/>
</launch>
```

Syntax: <remap from="/original_name" to="/new_name"/> — remap must be inside the <node> tag to apply only to that node, or inside <launch> to apply globally.

Roslaunch: `<include>` Tag

`<include>` embeds another launch file inside yours. This is how real robot packages are structured — each subsystem has its own launch file, and a top-level file includes them all.

```
<!-- robot_bringup.launch — top-level launch file -->
<launch>

  <!-- Include the sensor stack from another package -->
  <include file="$(find sensors_pkg)/launch/sensors.launch"/>

  <!-- Include the navigation stack -->
  <include file="$(find nav_pkg)/launch/navigation.launch"/>

  <!-- Include our custom nodes -->
  <include file="$(find my_robot)/launch/controllers.launch"/>

</launch>
```

`$(find package_name)` resolves to the absolute path of a ROS package — use it instead of hardcoded paths!

Roslaunch: `<param>` and `<rosparam>`

Load parameters directly inside your launch file — no separate terminal commands needed:

`<param>` — set individual values

```
<launch>
  <!-- Set a global parameter -->
  <param name="max_speed" value="1.5"/>

  <!-- Private param inside a node -->
  <node pkg="my_pkg" ...>
    <param name="rate" value="10"/>
  </node>
</launch>
```

`<rosparam>` — load from YAML file

```
<launch>
  <!-- Load entire YAML config -->
  <rosparam
    file="$(find my_pkg)/config/params.yaml"
    command="load"/>

  <!-- Or dump params to file on exit -->
  <rosparam
    file="$(find my_pkg)/config/saved.yaml"
    command="dump"/>
</launch>
```

`<param>` = one value at a time | `<rosparam>` = load/dump entire YAML files | Both are set BEFORE nodes start

Roslaunch: <arg> Tag

<arg> lets you define variables in the launch file that can be overridden from the command line — making your launch files reusable and configurable.

```
<!-- my_robot.launch -->
<launch>
  <!-- Declare args with defaults -->
  <arg name="update_period" default="10"/>
  <arg name="robot_model" default="burger"/>
  <arg name="use_sim" default="false"/>

  <!-- Use args with $(arg name) syntax -->
  <param name="timing" value="$(arg update_period)"/>
  <param name="model" value="$(arg robot_model)"/>
</launch>
```

Override from terminal:

```
# Use defaults
$ roslaunch my_pkg
my_robot.launch

# Override args
$ roslaunch my_pkg
my_robot.launch \
  update_period:=30 \
  robot_model:=waffle \
  use_sim:=true
```

Roslaunch: Complete Tag Reference

<code><launch></code>	Root element — wraps all other tags
<code><node></code>	Run a node: pkg, type, name, output, respawn, ns
<code><group></code>	Group nodes under a shared namespace or condition
<code><param></code>	Set a single parameter (name + value)
<code><rosparam></code>	Load/dump entire YAML config files
<code><arg></code>	Define a launch-time variable with optional default
<code><remap></code>	Rename a topic/service used by a node
<code><include></code>	Embed another .launch file
<code><machine></code>	Target a remote machine for node execution
<code><env></code>	Set environment variables (PATH, IP, etc.)
<code><test></code>	Like <code><node></code> but used for unit/integration tests

Hands-on Exercise 1: Robot Speed Controller

CLASS 1

Goal: Control turtlesim's speed using ROS parameters — change speed without restarting the node!

Step 1 — Write a publisher node that reads speed from a parameter:

```
import rospy
from geometry_msgs.msg import Twist
rospy.init_node('speed_controller')
pub = rospy.Publisher('/turtle1/cmd_vel', Twist,
queue_size=10)
rospy.set_param('/turtle_speed', 1.0) # default speed
rate = rospy.Rate(10)
while not rospy.is_shutdown():
    speed = rospy.get_param('/turtle_speed', 1.0)
    msg = Twist(); msg.linear.x = speed; pub.publish(msg)
    rate.sleep()
```

Step 2 — Run it:

```
$ roscore
$ rosrunc turtlesim turtlesim_node
$ rosrunc my_pkg speed_controller.py
```

Step 3 — Change speed LIVE:

```
$ rosparam set /turtle_speed 2.5
$ rosparam set /turtle_speed 0.3
```

Hands-on Exercise 2: Two Turtles, Two Conversations

Goal: Use roslaunch namespaces to run two isolated turtlesim controllers. First observe the cross-subscription bug, then fix it.

Step 1 — Write broken.launch (no namespace):

```
<launch>
  <node pkg="turtlesim" type="turtlesim_node"
name="turtle1"/>
  <node pkg="turtlesim" type="turtlesim_node"
name="turtle2"/>
  <node pkg="my_pkg" type="controller.py"
name="ctrl1"/>
  <node pkg="my_pkg" type="controller.py"
name="ctrl2"/>
</launch>
```

Step 2 — Write fixed.launch (with namespaces):

```
<launch>
  <group ns="robot1">
    <node ... name="turtlesim"/>
    <node ... name="controller"/>
  </group>
  <group ns="robot2">
    <node ... name="turtlesim"/>
    <node ... name="controller"/>
  </group>
</launch>
```

Step 3 — Run rqt_graph before and after to visualise the difference. Each group should be fully isolated!

Roslaunch

- Example Repository
 - https://github.com/ROBOTIS-GIT/ros_tutorials
- If you want to view or download directly from the web, but want to apply it right away, you can copy and build the repository with the following command:

```
$ cd ~/catkin_ws/src
```

```
$ git clone c
```

```
$ cd ~/catkin_ws
```

```
$ catkin_make
```




Thank you!

